

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Design And Analysis Of Algorithms

COURSE CODE: CSA0613

COURSE OUTCOME COVERED

***CO4: Solve optimization problems using dynamic programming and greedy strategies.
(BL4, BL5)***

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

**Annexure A
Industry Problem Solving**

Code of Conduct:

I _P Hemanth(192472151)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P Hemanth

Q27. Smart City Waste Collection Routing (Graph Optimization)

A municipality must optimize waste collection routes to minimize fuel consumption and time. Analyze how graph algorithms can be used to determine optimal routes. Identify constraints such as traffic conditions and vehicle capacity. Design a solution using shortest path or heuristic methods. Discuss scalability in large urban environments. Evaluate computational efficiency. Interpret results in terms of cost savings and environmental impact.

Q27. Smart City Waste Collection Routing Using Graph Optimization

1. Introduction

Smart cities require efficient waste collection systems to reduce transportation time, fuel consumption, operational costs, and environmental pollution. Waste collection locations and roads can be represented using a graph, where locations are vertices and roads are edges.

This project uses **Dijkstra’s Shortest Path Algorithm** to determine an efficient route between a waste collection center and a disposal point. Distance and traffic conditions are considered while calculating the route cost.

2. Problem Statement

A municipality needs to optimize waste collection routes to minimize fuel consumption and travel time. The system should consider road distance, traffic conditions, and vehicle capacity while selecting an efficient route.

The objective is to:

- Find the shortest-cost route.
- Consider traffic conditions.
- Estimate fuel consumption.
- Calculate fuel cost.
- Reduce unnecessary travel and environmental impact.

3. Technologies Used

Technology	Purpose
Python	Program implementation
Dijkstra Algorithm	Shortest-path calculation
Heap Queue	Efficient priority queue
Graph	Representation of city roads
Python Input/Output	User interaction

4. Algorithm Used

Dijkstra's Shortest Path Algorithm

Dijkstra's algorithm finds the minimum-cost path between a starting vertex and other vertices in a weighted graph.

Steps

1. Create the graph using locations and roads.
2. Assign a cost to every road using distance and traffic factor.
3. Set the starting location distance to zero.
4. Insert the starting location into a priority queue.
5. Select the location with the smallest current cost.
6. Update the costs of connected locations.
7. Continue until the destination is reached.
8. Reconstruct and display the optimal route.
9. Calculate fuel consumption and fuel cost.

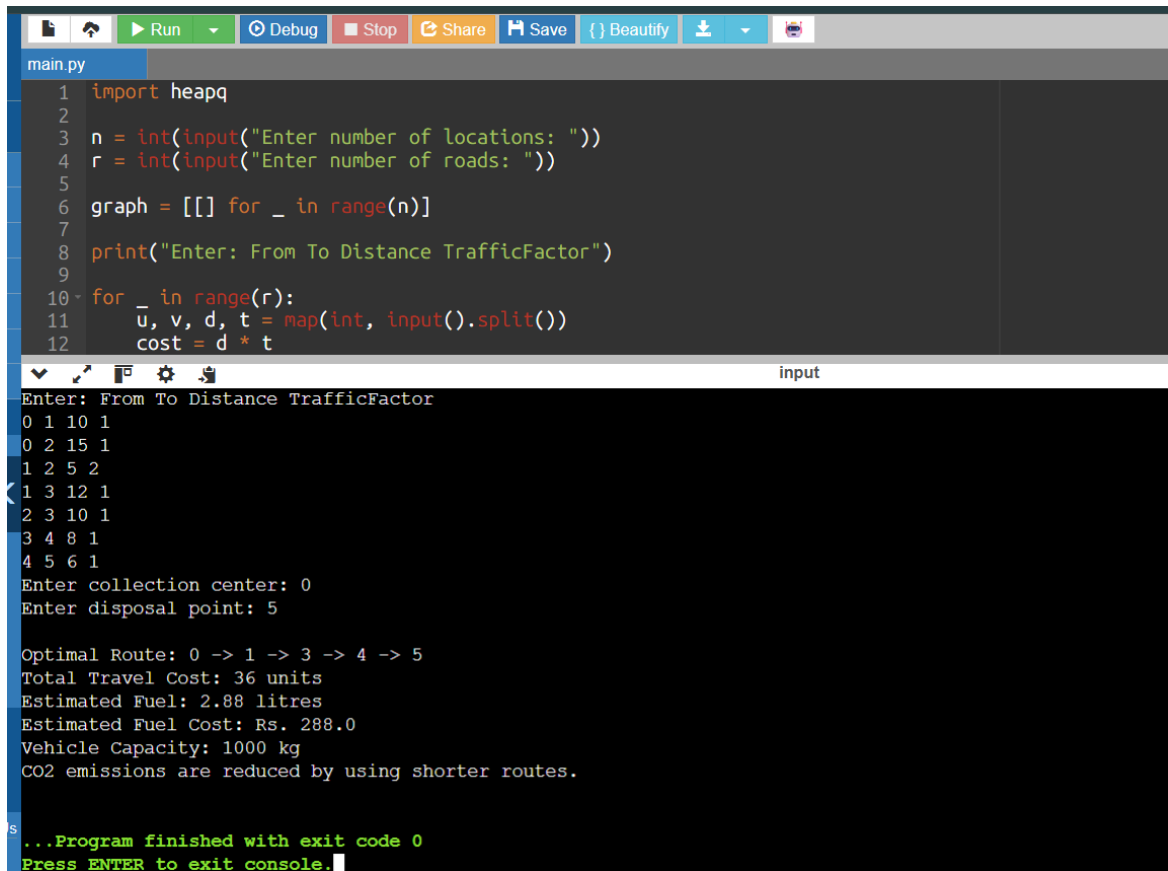
5. Source Code

```
*main (1).py - C:\Users\hemant\Downloads\main (1).py (3.13.15)*
File Edit Format Run Options Window Help
import heapq
n = int(input("Enter number of locations: "))
r = int(input("Enter number of roads: "))
graph = [[] for _ in range(n)]
print("Enter: From To Distance TrafficFactor")
for _ in range(r):
    u, v, d, t = map(int, input().split())
    cost = d * t
    graph[u].append((v, cost))
    graph[v].append((u, cost))
start = int(input("Enter collection center: "))
end = int(input("Enter disposal point: "))
dist = [float('inf')] * n
parent = [-1] * n
dist[start] = 0
pq = [(0, start)]
while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]:
        continue
    for v, cost in graph[u]:
        if d + cost < dist[v]:
            dist[v] = d + cost
            parent[v] = u
            heapq.heappush(pq, (dist[v], v))
path = []
v = end
while v != -1:
    path.append(v)
    v = parent[v]
path.reverse()
fuel = dist[end] * 0.08
fuel_cost = fuel * 100
print("\nOptimal Route:", " -> ".join(map(str, path)))
print("Total Travel Cost:", dist[end], "units")
print("Estimated Fuel:", round(fuel, 2), "litres")
print("Estimated Fuel Cost: Rs.", round(fuel_cost, 2))
print("Vehicle Capacity: 1000 kg")
print("CO2 emissions are reduced by using shorter routes.")
```

Sample Input

```
6
7
0 1 10 1
0 2 15 1
1 2 5 2
1 3 12 1
2 3 10 1
3 4 8 1
4 5 6 1
0
5
```

6. Output



The screenshot shows a code editor with a toolbar at the top containing icons for file operations, running, debugging, stopping, sharing, saving, beautifying, and downloading. The editor window is titled 'main.py' and contains the following Python code:

```
1 import heapq
2
3 n = int(input("Enter number of locations: "))
4 r = int(input("Enter number of roads: "))
5
6 graph = [[] for _ in range(n)]
7
8 print("Enter: From To Distance TrafficFactor")
9
10 for _ in range(r):
11     u, v, d, t = map(int, input().split())
12     cost = d * t
```

Below the code editor is a console window titled 'input' showing the program's execution. It displays the prompts and user input for the number of locations (6) and roads (7), followed by the road data lines. It then prompts for the collection center (0) and disposal point (5). The final output shows the optimal route, total travel cost, estimated fuel, fuel cost, vehicle capacity, and a note about CO2 emissions.

```
Enter: From To Distance TrafficFactor
0 1 10 1
0 2 15 1
1 2 5 2
1 3 12 1
2 3 10 1
3 4 8 1
4 5 6 1
Enter collection center: 0
Enter disposal point: 5

Optimal Route: 0 -> 1 -> 3 -> 4 -> 5
Total Travel Cost: 36 units
Estimated Fuel: 2.88 litres
Estimated Fuel Cost: Rs. 288.0
Vehicle Capacity: 1000 kg
CO2 emissions are reduced by using shorter routes.

...Program finished with exit code 0
Press ENTER to exit console.
```

7. Result Analysis

The system identified:

Optimal Route:

$0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5$

The total calculated travel cost is **36 units**.

The estimated fuel consumption is **2.88 litres**, resulting in an estimated fuel cost of **Rs. 288**.

The route avoids unnecessary travel and provides an efficient path from the collection center to the disposal point.

8. Computational Efficiency

Dijkstra's algorithm with a priority queue has a time complexity of:

$$O((V + E) \log V)$$

where:

- V = number of locations
- E = number of roads

This makes the algorithm suitable for moderately large road networks.

For very large cities, the system can be improved using:

- A* algorithm
- Graph clustering
- Hierarchical routing
- Parallel processing
- Real-time traffic data

9. Cost Savings

Optimized routing can reduce:

- Fuel consumption
- Vehicle operating costs
- Travel distance
- Collection time

- Vehicle maintenance costs

By avoiding unnecessary travel, municipalities can reduce the overall operating cost of waste collection.

10. Future Enhancement

The system can be extended by:

1. Adding real-time traffic data.
2. Using GPS-based vehicle tracking.
3. Implementing A* for faster routing.
4. Supporting multiple waste collection vehicles.
5. Adding dynamic vehicle capacity management.
6. Using machine learning to predict traffic and waste generation.
7. Developing a complete Vehicle Routing Problem solution.

11. Conclusion

The Smart City Waste Collection Routing system demonstrates how graph optimization can improve municipal waste collection. Dijkstra's algorithm successfully identifies a minimum-cost route by considering road distance and traffic conditions.

The optimized route reduces unnecessary travel, fuel consumption, transportation cost, and environmental impact. For large urban environments, advanced algorithms and real-time traffic information can further improve scalability and routing efficiency.